Lecture-8

JavaScript Fundamentals

var, let, const · Functions · Primitive vs Non-Primitive types



var/let/const



Functions



Primitives



Non-Primitives

TODAY

What we're covering



Declaring variables

var, let, and const — how they differ



Writing functions

Reusable blocks of code you can call anytime



Primitive types

String, Number, Boolean, and more — copied by value



Non-primitive types

Object, Array, Function — copied by reference

VARIABLES

var — the original way

```
var name = "Aman";
var name = "Priya";    // re-declaring is fine
console.log(name);     // "Priya"
```

Function-scoped — visible everywhere inside the function it's declared in, not just a block

Can be re-declared with the same name without any error

Hoisted and initialized to undefined — reading it before the line runs gives undefined, not an error

VARIABLES

let — block-scoped and reassignable

```
let age = 21;
age = 22;                  // ✅ reassigning is fine
let age = 25;               // ❌ SyntaxError: already declared
```

Block-scoped — only exists inside the { } it was declared in (if, for, function, etc.)

CAN be reassigned, but CANNOT be re-declared in the same scope

Hoisted, but stuck in the "Temporal Dead Zone" until its line runs — using it earlier throws an error

VARIABLES

const — block-scoped and locked

```
const PI = 3.14;
PI = 3.14159;   // ❌ TypeError: Assignment to constant variable
const PI;       // ❌ SyntaxError: Missing initializer
```

Block-scoped, just like let

MUST be given a value at the moment it's declared — no declaring now, assigning later

Can NEVER be reassigned after that — but if it holds an object/array, the contents can still change

VARIABLES

var vs let vs const — at a glance

	var	let	const
Scope	Function	Block	Block
Re-declare?	Yes	No	No
Reassign?	Yes	Yes	No
Must initialize?	No	No	Yes

FUNCTIONS

A reusable block of code

```
function greet(name) {
  return "Hello, " + name + "!";
}

console.log(greet("Aman"));   // "Hello, Aman!"
console.log(greet("Priya"));  // "Hello, Priya!"
```

Write the logic ONCE inside the function, then call greet(...) as many times as you want with different inputs — no copy-pasting the same code again and again.

FUNCTIONS

The anatomy of a function

`function greet(...)`	Declares a function named greet
`name`	Parameter — a placeholder for the value that will be passed in
`greet("Aman")`	Calling the function — "Aman" here is the argument
`return ...`	Sends a value back out to wherever the function was called

DATA TYPES

Primitive vs Non-Primitive: the big idea



Primitive

Stored by VALUE

Copying it makes a completely independent copy. Changing the copy never affects the original.



Non-Primitive

Stored by REFERENCE

Copying it just copies the address/pointer. Both copies point to the SAME data — changing one changes the other.

PRIMITIVE TYPES

The 7 primitive types in JavaScript

String

```
"hello"
```

Number

```
42, 3.14
```

Boolean

```
true, false
```

undefined

```
let x;
```

null

```
let y = null;
```

Symbol

```
Symbol('id')
```

BigInt

```
123n
```

NON-PRIMITIVE TYPES

Everything else is an Object

Object `{ name: "Aman", age: 21 }`

Array `[1, 2, 3, 4]`

Function `function greet() {...}`

Arrays and Functions are technically special kinds of Objects — that's why typeof [] and typeof (function(){}) both report as object-like behavior underneath.

LIVE EXAMPLE

Value vs Reference, side by side

```
let a = 10;
let b = a;
b = 20;

console.log(a);  // 10
console.log(b);  // 20
```

```
let obj1 = { value: 10 };
let obj2 = obj1;
obj2.value = 20;

console.log(obj1.value);  // 20
console.log(obj2.value);  // 20
```

Left (primitive): b gets its own independent copy of 10 — changing b never touches a.
Right (non-primitive): obj2 doesn't get a new object — it points to the SAME object as obj1. Changing obj2.value also changes what obj1.value reads as.

RECAP

What we covered today



var / let / const

Scope, redeclare, and reassignment rules differ



Functions

Write once, call many times, with parameters



Primitive types

String, Number, Boolean... copied by value



Non-Primitive types

Object, Array, Function... copied by reference

Thank You !